The reorganization heuristic is then applied if the check's condition is satisfied; a good understanding of k-means is needed to get into the details of this heuristic, so will skip it in this context, and we encourage readers to refer to chapter 12 for a description of this clustering algorithm. Here, we will use an example to illustrate how the heuristic works.

In the example, the heuristic is called on node S1 and the clustering is run on its grandchildren, points A-K. As mentioned, these could as well be centroids of other internal nodes, and the algorithm wouldn't change.

The result is shown in the bottom half of figure 10.28: you might wonder why now there are only 4 children in node S1; even if k-means was called with k, the number of initial clusters, equal to 5 (the number of S1's children), this clustering algorithm can output fewer than k clusters: if at any point, during its second step, points assignment, one of the centroids is not assigned any point, it just gets removed and the algorithm continues with one less cluster.

Both the check and the reorganization heuristic are resource-consuming, the latter in particular requires O(jM2k) comparisons/assignments, if j is the number of maximum iterations we use in k-means. Therefore, in the original paper it was recommended to check the overlap situation after splitting nodes, but to apply the reorganization infrequently.

We can also easily run the check also when an entry is moved across siblings, while it becomes less intuitive when we merge two nodes: in that case, we could always check all pairs of the merged node's sibling, or – to limit the cost - just sample some pairs.